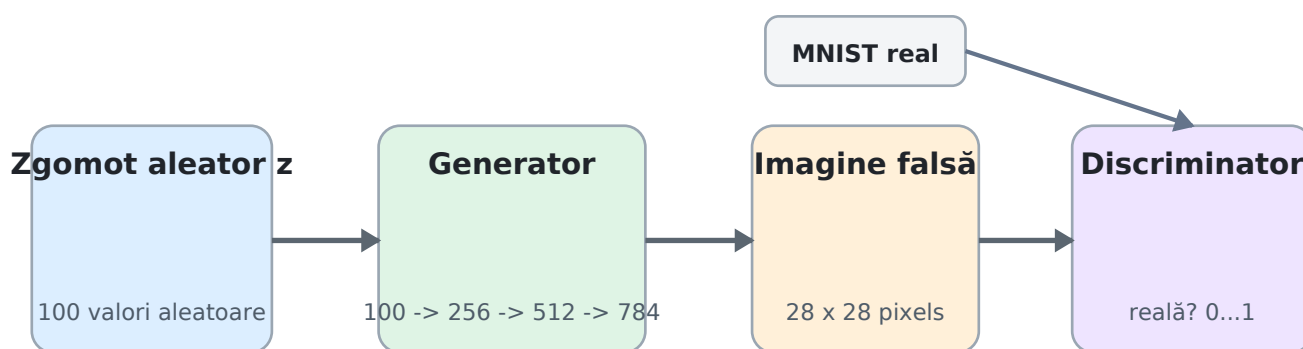


Cum lucrează un GAN

Explicație simplă, pas cu pas, cu exemplul MNIST



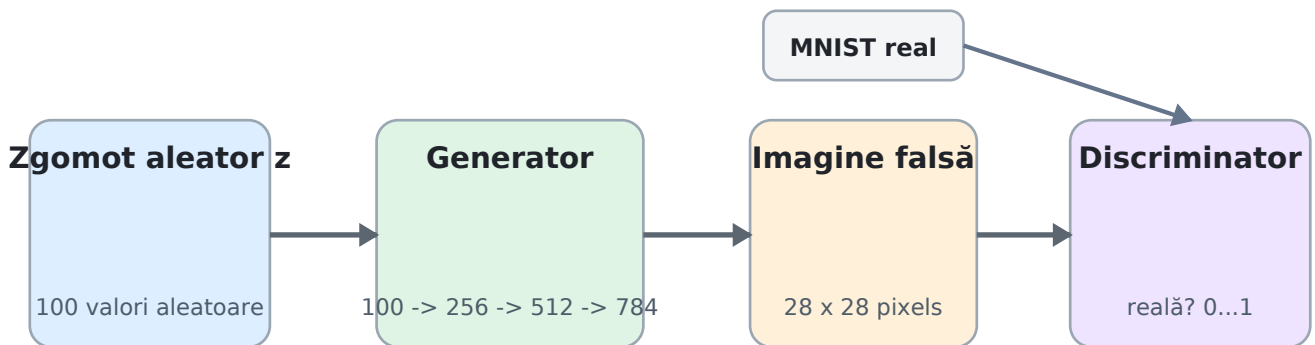
Generatorul încearcă să păcălească Discriminatorul; Discriminatorul încearcă să detecteze falsurile.

Ideea de bază: un Generator creează exemple false, iar un Discriminator încearcă să le deosebească de exemplele reale. Prin această competiție, Generatorul învață să producă exemple tot mai realiste.

1. Cele două rețele dintr-un GAN

Generatorul (G) creează date false. În exemplul MNIST, el încearcă să producă imagini de 28 x 28 pixeli care să semene cu cifre scrise de mână.

Discriminatorul (D) primește atât imagini reale din MNIST, cât și imagini create de Generator. El returnează un număr între 0 și 1: cât de probabil este ca imaginea să fie reală.



Generatorul încearcă să păcălească Discriminatorul; Discriminatorul încearcă să detecteze falsurile.

Analogie simplă

Generator	falsificator care încearcă să creeze bancnote cât mai credibile
Discriminator	expert care încearcă să depisteze falsurile

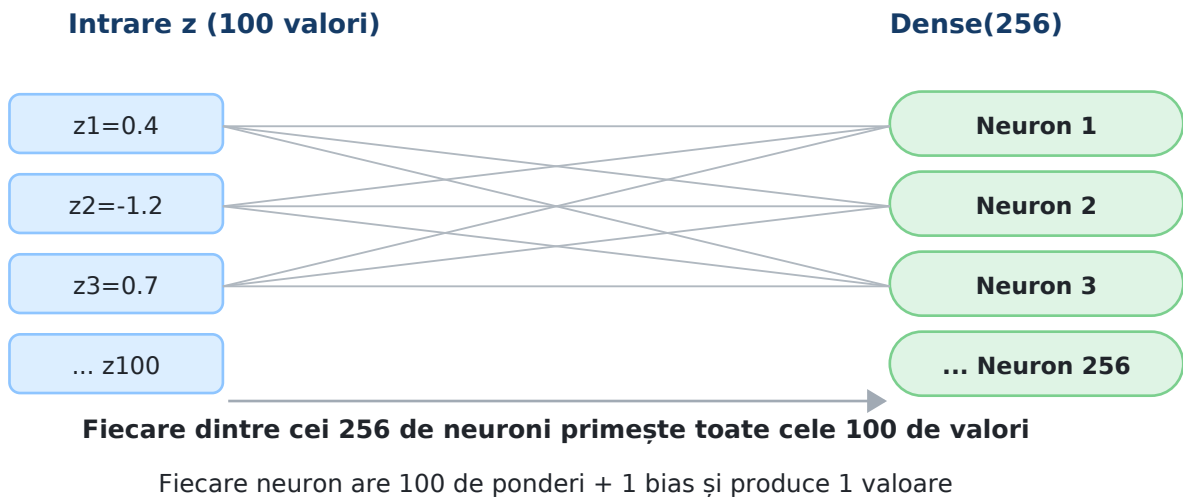
2. Ce este vectorul de zgomot z?

Generatorul nu primește direct o cifră. El primește un vector de numere aleatoare, de exemplu:

$$\mathbf{z} = [0.4, -1.2, 0.7, \dots]$$

În laborator putem folosi 100 de valori. Acest vector este doar o "sămânță" (seed). El nu înseamnă "7", "3" sau altă cifră.

Cum devin 100 de valori -> 256 de valori?



Fiecare dintre cei 256 de neuroni primește toate cele 100 de valori. Fiecare neuron are propriile 100 de ponderi și propriul bias. De aceea produce un rezultat diferit.

$$\text{Pentru neuronul } j: \text{output}_j = \text{activation}(w_1*z_1 + w_2*z_2 + \dots + w_{100}*z_{100} + \text{bias}_j)$$

3. Un exemplu numeric foarte simplu

Ca să vedem mecanismul, luăm doar 3 valori de intrare și calculăm 2 neuroni. În realitate sunt 100 de intrări și 256 de neuroni.

Intrare	Ponderi - Neuron 1	Ponderi - Neuron 2
$z1 = 0.4$	0.5	-0.2
$z2 = -1.2$	-0.3	0.6
$z3 = 0.7$	0.8	0.1
bias	0.1	-0.05

Neuron 1: $0.4 \times 0.5 + (-1.2) \times (-0.3) + 0.7 \times 0.8 + 0.1 = 1.22$

Neuron 2: $0.4 \times (-0.2) + (-1.2) \times 0.6 + 0.7 \times 0.1 - 0.05 = -0.78$

După funcția de activare, obținem două valori noi. Cu 256 de neuroni obținem 256 de valori noi.

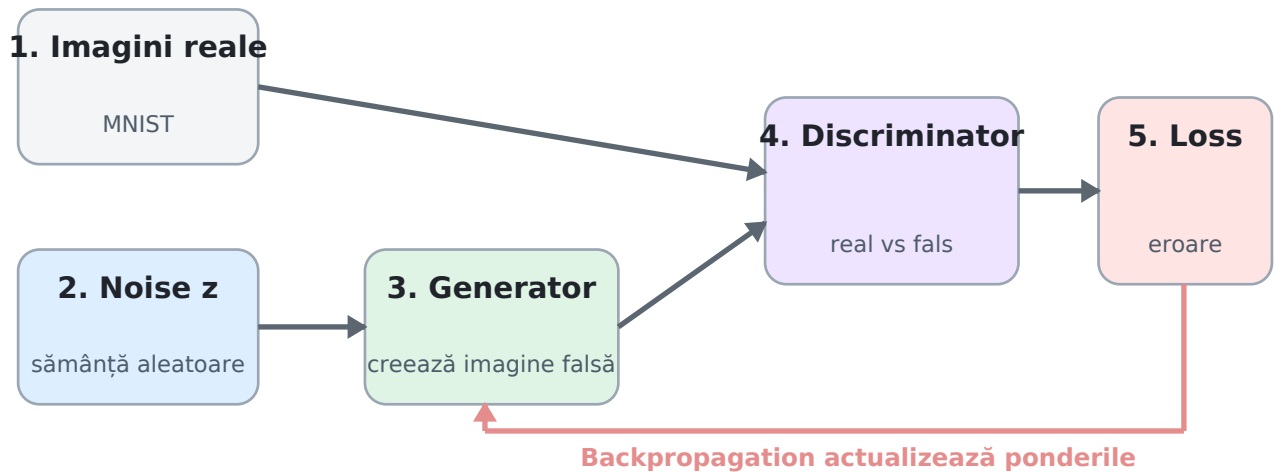
Important: 100 -> 256 nu înseamnă că datele sunt copiate sau augmentate. Înseamnă că 256 de neuroni fac 256 de calcule diferite folosind aceleași 100 de intrări.

Generatorul complet din laborator

**100 valori (z) -> 256 features -> 512 features -> 784 pixeli -> reshape
28 x 28**

4. Cum învață Generatorul sa producă cifre?

La început, ponderile Generatorului sunt aproape aleatoare. De aceea, imaginile produse arată ca zgomot.



- Pasul 1. Luăm imagini reale din MNIST.
- Pasul 2. Generatorul produce imagini false din vectori z .
- Pasul 3. Discriminatorul învață să spună: real = 1, fake = 0.
- Pasul 4. Generatorul încearcă să producă imagini pe care Discriminatorul să le creadă reale.
- Pasul 5. Eroarea este propagată înapoi (backpropagation), iar ponderile sunt actualizate.
- Pasul 6. Procesul se repetă de mii de ori.

Generatorul nu primește formula distribuției MNIST. El o învață indirect din feedbackul Discriminatorului.

5. Ce se întâmplă după antrenare?

După antrenare, diferiți vectori z produc imagini diferite, dar toate tind să semene cu cifre din MNIST.

La început	$z \rightarrow$ Generator $\rightarrow$ imagine aproape aleatoare
După multe iterații	$z \rightarrow$ Generator $\rightarrow$ cifră recognoscibilă
Alt z	poate produce altă cifră sau alta formă a aceleiași cifre

Aplicații tipice ale GAN-urilor

- generarea de imagini sintetice;
- data augmentation - adăugarea de exemple sintetice în setul de antrenare;
- image-to-image translation (de exemplu zi $\rightarrow$ noapte);
- super-resolution și restaurare de imagini;
- generarea de date sintetice pentru cercetare și testare.

De reținut:

- 1) z este doar o sămânță aleatoare.
- 2) Generatorul transformă z prin straturi Dense în pixeli.
- 3) Discriminatorul oferă feedback despre realism.
- 4) Backpropagation modifică ponderile ambelor rețele.
- 5) Prin competiție, Generatorul învață să producă exemple care seamănă cu datele reale.